Below is a **complete real-world Java version guide from Java 1.0 → Java 21**, including **what it's used for today**, and **when you should avoid it** (in real software development).

✅ Tip: In modern development, you should mainly choose between **Java 8 / 11 / 17 / 21** (LTS versions).
Others are mostly for legacy systems or short-lived feature testing.

✅ Java Versions (1 → 21) + Real-Time Usage Guide (Table)

Java Version	Release Type	Real-time Usage (Today)	Use Cases	When NOT to Use
Java 1.0	Legacy	❌ Not used	Historical / learning only	Never for real projects
Java 1.1	Legacy	❌ Not used	Old systems only	Avoid completely
Java 1.2	Legacy	❌ Not used	Old enterprise apps	Avoid completely
Java 1.3	Legacy	❌ Not used	Old bank/insurance apps	Avoid completely
Java 1.4	Legacy	❌ Not used	Old legacy systems	Avoid completely
Java 5 (1.5)	Legacy	❌ Rare	Very old enterprise apps	Avoid (no support, insecure)
Java 6 (1.6)	Legacy	❌ Rare	Very old server software	Avoid (unsupported)
Java 7 (1.7)	Legacy	❌ Rare	Some old Android/enterprise	Avoid unless forced
Java 8 (1.8)	**LTS**	✅ Very common	Spring Boot legacy + many companies	Don't choose for new projects if 17/21 possible
Java 9	Short-term	⬜ Rare	Module system testing	Don't use (not LTS)
Java 10	Short-term	⬜ Rare	Local var type (var) testing	Don't use (not LTS)
Java 11	**LTS**	✅ Very common	Enterprise APIs, Spring Boot apps	Not best if you can use 17/21

Java 12	Short-term	□ Rare	Feature preview testing	Don't use (not LTS)
Java 13	Short-term	□ Rare	Developer testing	Don't use (not LTS)
Java 14	Short-term	□ Rare	Switch expressions preview	Don't use (not LTS)
Java 15	Short-term	□ Rare	Text blocks feature	Don't use (not LTS)
Java 16	Short-term	□ Rare	Records improvements	Don't use (not LTS)
Java 17	**LTS**	✅ Most recommended	Modern Spring Boot, Microservices, Cloud	Don't use only if org requires 8/11
Java 18	Short-term	□ Rare	New features evaluation	Don't use (not LTS)
Java 19	Short-term	□ Rare	Virtual threads preview	Don't use in production (unless company policy allows)
Java 20	Short-term	□ Rare	Continued previews	Don't use (not LTS)
Java 21	**LTS**	✅ Best for new apps	High performance backend + modern apps	Don't use only if framework/org doesn't support yet

---

✅ Practical Recommendation (Real World)

✅ Best Version Choices for Production

Scenario	**Best Java Version**
New project (2026 modern backend)	✅ **Java 21 (LTS)**
Company wants stable & widely supported	✅ **Java 17 (LTS)**
Enterprise legacy but still supported	✅ **Java 11 (LTS)**
Very old systems (banks / govt / old Spring Boot)	✅ **Java 8 (LTS)**

✅ Why LTS Matters (Short and Clear)

✅ LTS (Long-Term Support)

Used in real companies because:

- Long security updates
- Best framework support (Spring Boot, Hibernate, Kubernetes tools)
- Stable builds for production

✅ LTS versions: **8, 11, 17, 21**

❌ Non-LTS (Short-Term)

- Used only for testing features
- Upgrade needed quickly
- Not stable long-term for enterprise

✅ Best Real-time Use Cases by Java Version

Java Version	Most Practical Use Case
Java 8	Legacy Spring Boot apps, old company systems
Java 11	Enterprise API systems, stable cloud services
Java 17	Cloud native microservices, modern backend development
Java 21	High-performance backend + virtual threads + modern Java development

---

If you want, I can also provide:
✅ **Java 8 vs 11 vs 17 vs 21 detailed comparison table**,
✅ **Spring Boot compatibility matrix**,
✅ Which version is best for **Android**, **Microservices**, **Kubernetes**, **AWS** etc.

Below is the **most detailed real-time comparison table of Java 8 vs 11 vs 17 vs 21**, focusing on **what matters in software development today** (Spring Boot, performance, cloud, upgrades, and use cases).

✅ **Java 8 vs 11 vs 17 vs 21 (Detailed Comparison Table)**

Feature / Factor	Java 8 (LTS)	Java 11 (LTS)	Java 17 (LTS)	Java 21 (LTS)
Release Year	2014	2018	2021	2023
LTS Support Type	Legacy LTS	Stable LTS	Modern LTS	Latest LTS
Best For	Old enterprise apps	Enterprise apps	Modern production systems	New gen high-performance systems
Real Industry Usage (2026)	✅ Many legacy projects	✅ Very common	✅ Most recommended	✅ Fast adoption (new projects)
Spring Boot Compatibility	✅ Works (2.x mostly)	✅ Works (2.x / 3 partial)	✅ Best for Spring Boot 3.x	✅ Best for Spring Boot 3.2+
Jakarta EE Support	❌ Old javax	⬜ Mixed	✅ Full (Jakarta)	✅ Full (Jakarta)
JDK Installation	Usually Oracle/OpenJDK	OpenJDK widely used	OpenJDK widely used	OpenJDK widely used
Performance	Good	Better than 8	Much better	Best overall
Garbage Collection	G1 available but older	Improved G1	Improved GC tuning	Latest GC improvements
Startup Time (Microservices)	Slower	Better	Faster	Fastest
Memory Usage	Higher	Lower	Better optimized	Best optimized
Security	❌ Weak vs modern standards	✅ Better security	✅ Strong	✅ Strongest
Language Features	Lambda, Stream	var in lambda, more APIs	Records, Sealed classes	Pattern matching upgrades
JVM Feature Updates	Basic	Better runtime	Big JVM improvements	Latest runtime improvements

Virtual Threads	❌ No	❌ No	❌ No	✅ **Yes (Project Loom)**
Concurrency Scalability	Thread heavy	Thread heavy	Thread heavy	✅ Massive scalability
HTTP Client	❌ No built-in (use libs)	✅ New HttpClient	✅ Improved	✅ Improved
Docker / Cloud Ready	⚠ Not best	✅ Good	✅ Best	✅ Best
Kubernetes Microservices	⚠ okay	✅ good	✅ best	✅ best (cheap concurrency)
Tooling / IDE Support	✅ Full	✅ Full	✅ Full	✅ Full
Learning Curve	Easy	Easy	Medium	Medium
Upgrade Difficulty	Base	Easy from 8	Moderate	Moderate
Recommended for New Projects	❌ No	⚠ Only if required	✅ Yes	✅ Best choice
When NOT to Use	New projects	If you can use 17/21	Only if company stuck on 11	Only if framework/org doesn't support yet

---

✅ Feature Comparison (What Each Version Added)

Java Version	Major Additions (Real Use)
Java 8	✅ Lambda Expressions, Streams, Optional, Default Methods, Date/Time API
Java 11	✅ New HttpClient API, TLS improvements, performance upgrades, removed old modules
Java 17	✅ Records, Sealed Classes, strong encapsulation, faster JVM, long-term stability
Java 21	✅ **Virtual Threads**, Pattern Matching maturity, Record Patterns, huge concurrency improvement

---

✅ **Which One to Choose? (Real-Time Decision Table)**

Situation	Best Version
You are starting a brand new backend API / microservice	✅ **Java 21**
You want safest enterprise option with high support	✅ **Java 17**
Your company uses old Spring Boot 2.x / old infra	✅ **Java 11**
Banking / Government legacy systems (very old)	✅ **Java 8**
High concurrency apps (chat, messaging, realtime)	✅ **Java 21** (Virtual threads)
Job interviews / learning fundamentals	✅ Java 8 + 17

✅ **Summary (Quick Final Recommendation)**

✅ **Best overall for new development (2026): Java 21**
✅ **Most balanced & widely supported: Java 17**
✅ **Still common in enterprise: Java 11**
 Only for legacy systems: Java 8

If you want, I can also make:
✅ **Spring Boot 2.x vs 3.x Java compatibility matrix**
✅ A **migration plan: Java 8 → 11 → 17 → 21** with code examples and tools.